

LAPORAN PROJECT
SISTEM MANAJEMEN PASIEN KLINIK

Disusun untuk:

memenuhi tugas Ujian Akhir Semester Mata Kuliah Algoritma dan Struktur Data

Dosen Pengampu:

Taufik Ridwan, M.T.



Disusun Oleh:

Kelompok 8

Alfaridzh Maleo Pradayuda (2510631250008)

Calila Nafisa (2510631250056)

Mahilda Trijayanti (2510631250068)

Ramzi Arashyi A.K. (2510631250002)

PROGRAM STUDI SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS SINGAPERBANGSA KARAWANG

2026

KATA PENGANTAR

Puji syukur kami panjatkan kehadiran Allah SWT yang telah melimpahkan rahmat dan karunia-Nya sehingga kami dapat menyelesaikan laporan project ini dengan judul "Sistem Manajemen Pasien Klinik Berbasis Java" tepat pada waktunya.

Laporan ini disusun untuk memenuhi tugas Ujian Akhir Semester (UAS) mata kuliah Algoritma dan Struktur Data. Melalui project ini, kami berupaya mengimplementasikan konsep algoritma dan struktur data secara manual, meliputi operasi CRUD, searching, sorting, serta statistik data pada sebuah sistem manajemen pasien klinik berbasis console.

Kami menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari Bapak Taufik Ridwan, M.T. selaku dosen pengampu, demi perbaikan di masa yang akan datang.

Akhir kata, kami mengucapkan terima kasih kepada semua pihak yang telah membantu dalam proses penyusunan laporan ini. Semoga laporan ini dapat bermanfaat bagi pembaca.

Karawang, Juni 2026

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Teknologi informasi telah menjadi bagian integral dalam pengelolaan data di berbagai bidang, termasuk di sektor kesehatan. Sistem manajemen pasien yang efisien sangat dibutuhkan oleh klinik maupun rumah sakit untuk memastikan data pasien tersimpan, dikelola, dan dapat diakses dengan cepat dan akurat.

Dalam konteks mata kuliah Algoritma dan Struktur Data, pemahaman tentang bagaimana data diorganisasi dan dimanipulasi secara efisien merupakan hal yang fundamental. Pemilihan algoritma yang tepat, mulai dari pencarian (searching) hingga pengurutan (sorting), memiliki dampak signifikan terhadap performa sistem secara keseluruhan.

Proyek ini mengimplementasikan Sistem Manajemen Pasien Klinik/Rumah Sakit menggunakan bahasa pemrograman Java dengan tampilan antarmuka grafis (GUI) berbasis Java Swing. Sistem ini dirancang untuk mengelola data pasien secara menyeluruh, meliputi operasi CRUD (Create, Read, Update, Delete), pencarian berbasis Linear Search dan Binary Search, pengurutan menggunakan Bubble Sort dan Selection Sort, serta fitur statistik data dan persistensi melalui file teks.

Implementasi dilakukan secara manual tanpa memanfaatkan library eksternal untuk fungsi pencarian dan pengurutan, sehingga memastikan pemahaman yang mendalam terhadap konsep algoritma dan struktur data yang dipelajari.

1.2 Tujuan

Tujuan dari pembuatan proyek Sistem Manajemen Pasien ini adalah sebagai berikut:

- a) Mengimplementasikan struktur data array of records untuk menyimpan dan mengelola data pasien secara efisien.

- b) Menerapkan algoritma pencarian Linear Search untuk pencarian data berdasarkan nama pasien.
- c) Menerapkan algoritma pencarian Binary Search (setelah data terurut) untuk pencarian data berdasarkan ID pasien.
- d) Mengimplementasikan algoritma Bubble Sort untuk mengurutkan data berdasarkan ID secara ascending.
- e) Mengimplementasikan algoritma Selection Sort untuk mengurutkan data berdasarkan nama secara alfabetis.
- f) Mengimplementasikan pengurutan descending berdasarkan total biaya pasien.
- g) Menyediakan fitur CRUD lengkap dengan soft delete menggunakan flag status aktif.
- h) Menyediakan fitur statistik data pasien termasuk total data, data aktif, dan total biaya.
- i) Mengimplementasikan mekanisme penyimpanan dan pemuatan data menggunakan file teks (data.txt).
- j) Membangun antarmuka pengguna grafis (GUI) yang intuitif menggunakan Java Swing.

1.3 Ruang Lingkup

Sistem yang dikembangkan mencakup pengelolaan data pasien pada klinik atau rumah sakit dengan fitur-fitur utama sebagai berikut: manajemen data pasien (ID, Nama, Umur, Poli, Status, Biaya Periksa, Biaya Obat), operasi CRUD berbasis GUI, algoritma searching dan sorting manual, serta penyimpanan data persisten ke file teks.

BAB II

ANALISIS KEBUTUHAN

2.1 Fitur Sistem

Sistem Manajemen Pasien yang diimplementasikan memiliki fitur-fitur berikut sesuai dengan ketentuan tugas:

2.1.1 CRUD Operations

- a) Tambah Data Baru: Menambahkan data pasien baru ke dalam array dengan validasi input.
- b) Tampilkan Semua Data: Menampilkan seluruh data pasien aktif dalam tabel GUI.
- c) Edit Data berdasarkan ID: Memperbarui informasi pasien berdasarkan ID yang dimasukkan.
- d) Hapus Data (Soft Delete): Menandai data sebagai tidak aktif (field aktif = false) tanpa menghapus data secara fisik dari array.

2.1.2 Searching Features

- a) Cari berdasarkan Nama (Linear Search): Menelusuri seluruh array satu per satu untuk menemukan pasien yang namanya mengandung kata kunci yang dimasukkan.
- b) Cari berdasarkan ID (Binary Search): Melakukan pencarian biner setelah data terurut berdasarkan ID untuk menemukan pasien dengan ID tertentu.
- c) Cari berdasarkan Poli (Kategori): Menampilkan semua pasien yang terdaftar pada poli tertentu.

2.1.3 Sorting Features

- 1) Sort berdasarkan ID (Bubble Sort - Ascending): Mengurutkan data pasien berdasarkan ID dari yang terkecil ke terbesar menggunakan algoritma Bubble Sort.

- 2) Sort berdasarkan Nama (Selection Sort - Alfabetis): Mengurutkan data pasien berdasarkan nama secara alfabetis menggunakan algoritma Selection Sort.
- 3) Sort berdasarkan Total Biaya (Descending): Mengurutkan data pasien berdasarkan total biaya dari yang terbesar ke terkecil.

2.1.4 Additional Features

- 1) Statistik Data: Menampilkan ringkasan data termasuk total data, data aktif, data terhapus, total biaya, dan rata-rata biaya.
- 2) Simpan/Load Data ke/dari File Teks: Data disimpan secara otomatis ke file data.txt dan dimuat kembali saat program dijalankan.
- 3) Update Biaya Otomatis: Setiap perubahan pada biaya pemeriksaan atau biaya obat secara otomatis memperbarui total biaya.

2.2 Struktur Data

Sistem ini menggunakan struktur data berikut:

2.2.1 Class Pasien (Record)

Setiap entitas pasien direpresentasikan oleh objek kelas Pasien dengan atribut-atribut sebagai berikut:

Atribut	Tipe Data	Keterangan
id	String	Identitas unik pasien (contoh: P001)
nama	String	Nama lengkap pasien
umur	int	Usia pasien dalam tahun
poli	String	Poli/departemen tujuan pasien (contoh: Umum, Gigi)
status	String	Status rawat (Rawat Jalan / Rawat Inap)
biayaPeriksa	double	Biaya pemeriksaan dalam rupiah
biayaObat	double	Biaya obat-obatan dalam rupiah
total	double	Total biaya (biayaPeriksa + biayaObat), dihitung otomatis

aktif	boolean	Flag soft delete (true = aktif, false = terhapus)
-------	---------	---

2.2.2 Array of Records

Data seluruh pasien disimpan dalam array bertipe Pasien[] dengan kapasitas maksimal 200 elemen. Variabel n digunakan untuk melacak jumlah data yang tersimpan saat ini.

```
Pasien[] data = new Pasien[200];
int n = 0; // jumlah data saat ini
```

Penggunaan array of records dipilih karena: (1) akses elemen dengan indeks memiliki kompleksitas $O(1)$, (2) sesuai dengan konsep struktur data yang dipelajari, dan (3) memungkinkan implementasi algoritma sorting dan searching secara manual.

BAB III

IMPLEMENTASI

3.1 CRUD Operations

A Tambah Data Pasien

Kode ini digunakan untuk menambahkan data pasien baru ke dalam array. Data seperti ID, nama, umur, poli, status, biaya periksa, dan biaya obat diambil dari form input. Setelah itu data disimpan ke dalam object Pasien lalu dimasukkan ke array data.

```
void tambahData() {
    String id = txtID.getText();
    String nama = txtNama.getText();
    int umur = Integer.parseInt(txtUmur.getText());
    String poli = txtPoli.getText();
    String status = txtStatus.getText();
    double bp = Double.parseDouble(txtPeriksa.getText());
    double bo = Double.parseDouble(txtObat.getText());

    data[n++] = new Pasien(id, nama, umur, poli, status, bp, bo);
    tampilData();
    JOptionPane.showMessageDialog(this, "Data berhasil ditambahkan!");
    clearForm();
}
```

B Tampil Data

Method ini berfungsi untuk menampilkan seluruh data pasien ke tabel GUI.

`model.setRowCount(0)` dipakai untuk mengosongkan isi tabel terlebih dahulu supaya data tidak dobel saat ditampilkan ulang.

Lalu program melakukan perulangan dari awal sampai jumlah data (n). Jika data pasien masih aktif (aktif = true), maka data tersebut akan dimasukkan ke tabel menggunakan `addRow()`.

```
void tampilData() {
    model.setRowCount(0);
    for (int i = 0; i < n; i++) {
        if (data[i].aktif) {
            model.addRow(new Object[]{
                data[i].id, data[i].nama, data[i].umur,
                data[i].poli, data[i].status, data[i].total
            });
        }
    }
}
```



```
}  
}
```

Jadi fungsi utama method ini adalah menampilkan data pasien yang masih aktif ke dalam tabel.

C Edit Data berdasarkan ID

Kode ini digunakan untuk mengubah data pasien yang sudah ada. Program akan mencari data berdasarkan ID yang diinput pada txtID. Jika ID ditemukan dan data masih aktif, maka data pasien seperti nama, umur, poli, dan status akan diperbarui sesuai input terbaru. Setelah itu method hitungTotal() dijalankan untuk menghitung ulang total biaya pasien.

```
void editData() {  
    String cari = txtID.getText();  
    for (int i = 0; i < n; i++) {  
        if (data[i].id.equalsIgnoreCase(cari) && data[i].aktif) {  
            data[i].nama = txtNama.getText();  
            data[i].umur = Integer.parseInt(txtUmur.getText());  
            data[i].poli = txtPoli.getText();  
            data[i].status = txtStatus.getText();  
            data[i].hitungTotal();  
            tampilData();  
            return;  
        }  
    }  
}
```

Kemudian tampilData() dipanggil agar perubahan langsung terlihat pada tabel, lalu muncul pesan bahwa data berhasil diupdate. Kalau ID tidak ditemukan, program akan menampilkan pesan “Data tidak ditemukan”

D Hapus Data

Method ini digunakan untuk menghapus data pasien dengan metode soft delete. Program mencari data berdasarkan ID pasien. Jika ditemukan, maka nilai aktif diubah menjadi false.

```
void hapusData() {  
    String cari = txtID.getText();  
    for (int i = 0; i < n; i++) {  
        if (data[i].id.equalsIgnoreCase(cari) && data[i].aktif) {  
            data[i].aktif = false;  
            tampilData();  
        }  
    }  
}
```

```

        return;
    }
}
}

```

Data sebenarnya tidak benar-benar dihapus dari array, tetapi hanya disembunyikan dari tampilan tabel. Setelah itu `tampilData()` dijalankan kembali agar data yang sudah dihapus tidak muncul lagi di tabel.

3.2 Searching

a) Linear Search berdasarkan Nama

Data yang ditampilkan meliputi ID, nama, umur, poli, status, dan total. Secara keseluruhan, method `cariNama()` berfungsi sebagai fitur pencarian data berdasarkan nama agar pengguna dapat menemukan data tertentu dengan lebih cepat dan praktis. Karena menggunakan `contains()`, pencarian tidak harus sama persis, cukup sebagian kata saja maka data yang sesuai tetap dapat ditemukan.

```

void cariNama() {
    String cari = txtCari.getText().toLowerCase();
    model.setRowCount(0);
    for (int i = 0; i < n; i++) {
        if (data[i].aktif && data[i].nama.toLowerCase().contains(cari)) {
            model.addRow(new Object[]{
                data[i].id, data[i].nama, data[i].umur,
                data[i].poli, data[i].status, data[i].total
            });
        }
    }
}
}

```

b) Binary Search berdasarkan ID

Kode tersebut digunakan untuk mencari data berdasarkan ID yang diinput pada `txtCari`, kemudian hasil yang cocok ditampilkan ke tabel melalui `model.addRow()`. Pencarian menggunakan `equalsIgnoreCase()` sehingga huruf besar dan kecil tidak memengaruhi hasil pencarian.

```

void cariID() {
    String cari = txtCari.getText();
    model.setRowCount(0);
    for (int i = 0; i < n; i++) {
        if (data[i].aktif && data[i].id.equalsIgnoreCase(cari)) {
            model.addRow(new Object[]{
                data[i].id, data[i].nama, data[i].umur,
                data[i].poli, data[i].status, data[i].total
            });
        }
    }
}

```

```

    });
  }
}
}

```

c) Pencarian berdasarkan Kategori/Poli

Fitur ini dirancang untuk memudahkan pencarian data pasien berdasarkan poliklinik atau kategori pelayanan kesehatan. Pengguna dapat memilih poli tertentu (misalnya: Poli Umum, Poli Gigi, Poli Anak, dll.) kemudian sistem akan menampilkan seluruh pasien yang terdaftar pada poli tersebut.

Fitur pencarian ini meningkatkan efisiensi dalam mengelola data pasien, terutama pada klinik atau rumah sakit yang memiliki banyak poliklinik. Selain itu, fitur ini dapat dikombinasikan dengan pencarian lain (seperti nama pasien) untuk hasil yang lebih akurat.

3.3 Sorting

1. Bubble Sort berdasarkan ID (Ascending)

Membandingkan dua elemen yang berdekatan dan menukarnya jika urutannya salah.

```

void sortID() {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (data[j].id.compareTo(data[j + 1].id) > 0) {
                Pasien temp = data[j];
                data[j] = data[j + 1];
                data[j + 1] = temp;
            }
        }
    }
}

```

2. Selection Sort berdasarkan Nama (Alphabetical)

Mencari elemen terkecil dari sisa array, lalu menukarnya dengan posisi saat ini.

```

void sortNama() {
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++) {
            if (data[j].nama.compareToIgnoreCase(data[min].nama) < 0) {
                min = j;
            }
        }
        Pasien temp = data[i];
        data[i] = data[min];
        data[min] = temp;
    }
}

```

3. Sort berdasarkan Total Biaya (Descending)

Data yang ditampilkan meliputi ID, nama, umur, poli, status, dan total. Secara keseluruhan, method cariNama() berfungsi sebagai fitur pencarian data berdasarkan nama agar pengguna dapat menemukan data tertentu dengan lebih cepat dan praktis. Karena menggunakan contains(), pencarian tidak harus sama persis, cukup sebagian kata saja maka data yang sesuai tetap dapat ditemukan.

```

void sortTotal() {
    Arrays.sort(data, 0, n, (a, b) -> Double.compare(b.total, a.total));
}

```

3.4 Fitur Tambahan

Update status & counter pasien

Fitur Update Status Pasien memungkinkan pengguna untuk mengubah status pasien secara dinamis, seperti dari 'Aktif' menjadi 'Selesai', 'Batal', atau 'Dirawat'. Fitur ini juga dilengkapi dengan counter pasien yang secara otomatis menghitung jumlah pasien aktif, pasien baru, dan pasien yang telah selesai ditangani. Tujuan fitur ini adalah untuk memudahkan monitoring kondisi pasien secara real-time.

Statistik data (total data, aktif, terhapus, total biaya, rata-rata biaya)

Fitur Statistik Data menyajikan ringkasan informasi penting mengenai data pasien. Statistik yang ditampilkan meliputi total data pasien, jumlah pasien aktif, pasien terhapus, total biaya keseluruhan, serta rata-rata biaya per pasien. Fitur ini

membantu pengguna dalam mengambil keputusan dan melihat gambaran keseluruhan data dengan cepat.

```
void statistik() {
    int aktif = 0, hapus = 0;
    double total = 0;
    for (int i = 0; i < n; i++) {
        if (data[i].aktif) {
            aktif++;
            total += data[i].total;
        } else {
            hapus++;
        }
    }
    JOptionPane.showMessageDialog(this,
        "===== STATISTIK =====\n\n" +
        "Total Data      : " + n +
        "\nData Aktif    : " + aktif +
        "\nData Terhapus  : " + hapus +
        "\nTotal Biaya    : " + total +
        "\nRata-rata      : " + (total / aktif)
    );
}
```

Simpan/Load data

Untuk menjaga agar data tidak hilang saat program ditutup, ditambahkan fitur Simpan (Save) dan Load data ke file teks (.txt). Fitur Simpan digunakan untuk menyimpan seluruh data pasien ke dalam file eksternal, sedangkan fitur Load digunakan untuk membaca dan memuat kembali data dari file tersebut. Dengan adanya fitur ini, data pasien dapat dibackup dan dipindahkan antar komputer dengan mudah.

BAB IV

ANALISIS KOMPLEKSITAS

4.1 Tabel Kompleksitas Algoritma

Fitur	Algoritma	Best Case	Average Case	Worst Case
Cari Nama	Linear Search	$O(1)$	$O(n)$	$O(n)$
Cari ID	Linear Search	$O(1)$	$O(n)$	$O(n)$
Sort ID	Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Sort Nama	Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Sort Total	Arrays.sort (TimSort)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Tambah Data	—	$O(1)$	$O(1)$	$O(1)$
Edit / Hapus	Linear Search	$O(1)$	$O(n)$	$O(n)$
Tampil Data	Traversal	$O(n)$	$O(n)$	$O(n)$
Statistik	Traversal	$O(n)$	$O(n)$	$O(n)$

4.2 Penjelasan Algoritma

1. Linear Search

Linear Search digunakan untuk mencari data pasien berdasarkan ID dengan cara memeriksa data satu per satu dari awal array sampai data ditemukan.

Algoritma ini dipilih karena sederhana dan mudah diterapkan pada array. Selain itu, data tidak harus dalam keadaan terurut sehingga pencarian bisa langsung dilakukan. Kelebihan dari Linear Search adalah mudah dibuat dan cocok untuk jumlah data kecil. Namun kekurangannya, proses pencarian menjadi lebih lambat jika jumlah data semakin banyak karena program harus memeriksa data satu per satu.

2. Binary Search

Binary Search digunakan untuk mencari data dengan lebih cepat pada data yang sudah diurutkan. Cara kerjanya yaitu membagi data menjadi dua bagian secara terus menerus sampai data ditemukan.

Algoritma ini dipilih karena memiliki proses pencarian yang lebih cepat dibanding Linear Search, terutama ketika jumlah data besar.

Kelebihannya adalah waktu pencarian lebih efisien dengan kompleksitas $O(\log n)$. Sedangkan kekurangannya, data harus diurutkan terlebih dahulu sebelum proses pencarian dilakukan.

3. Bubble Sort

Bubble Sort digunakan untuk mengurutkan data pasien dengan cara membandingkan dua data yang berdekatan lalu menukarnya jika urutannya salah. Algoritma ini dipilih karena mudah dipahami dan cocok digunakan untuk pembelajaran dasar algoritma sorting.

Kelebihan Bubble Sort adalah prosesnya sederhana dan mudah diimplementasikan. Namun kekurangannya, algoritma ini kurang efisien untuk data dalam jumlah besar karena membutuhkan banyak perbandingan dan pertukaran data.

4. Selection Sort

Selection Sort digunakan untuk mengurutkan data dengan mencari nilai terkecil atau terbesar lalu menempatkannya pada posisi yang sesuai.

Algoritma ini dipilih karena lebih sederhana dalam proses pertukaran data dibanding Bubble Sort.

Kelebihannya adalah jumlah pertukaran data lebih sedikit. Sedangkan kekurangannya, waktu proses tetap cukup lama jika digunakan pada data yang banyak karena kompleksitasnya $O(n^2)$.

5. CRUD (Create, Read, Update, Delete)

Fitur CRUD digunakan sebagai proses utama dalam pengelolaan data pasien.

- a) *Create* digunakan untuk menambah data pasien baru.
- b) *Read* digunakan untuk menampilkan data pasien.
- c) *Update* digunakan untuk mengubah data pasien.
- d) *Delete* digunakan untuk menghapus data menggunakan metode soft delete.

CRUD dipilih karena merupakan dasar utama dalam sistem pengelolaan data.

Kelebihannya adalah memudahkan pengelolaan data pasien secara lengkap. Sedangkan kekurangannya, proses edit dan hapus membutuhkan pencarian data terlebih dahulu sehingga waktu proses bergantung pada jumlah data yang ada

BAB V

PENGUJIAN

5.1 Skenario Pengujian CRUD

Pengujian CRUD dilakukan untuk memastikan fitur tambah, tampil, edit, dan hapus data pasien dapat berjalan dengan baik.

a. Pengujian Tambah Data

Pada pengujian ini, user mengisi data pasien seperti ID, nama, umur, poli, status, dan biaya pemeriksaan. Setelah tombol tambah ditekan, data berhasil masuk ke dalam tabel.

Hasil:

Data pasien berhasil ditambahkan dan tampil pada tabel sistem.

Form Pasien

ID

P050

Nama

Ramzi

Umur

19

Poli

Gigi

Status

Aktif

Biaya Periksa

2000000

Biaya Obat

2000000

Tambah

Edit

Pencarian

Menu

Hapus

Sort Nama

Sort Total

Statistik

ID	Nama	Umur	Poli	Status	Total
P001	Amir	25	Umum	Rawat Jalan	150000.0
P027	Asma	35	Saraf	Rawat Jalan	180000.0
P028	Bea	28	Umum	Rawat Jalan	200000.0
P002	Bulan	45	Gigi	Rawat Jalan	180000.0
P029	Cahyo	43	Jantung	Rawat Inap	550000.0
P003	Citra	30	Anak	Rawat Jalan	200000.0
P004	Dewi	28	Umum	Rawat Jalan	200000.0
P030	Dina	20	Anak	Rawat Jalan	200000.0
P005	Eka	35	Saraf	Rawat Jalan	180000.0
P006	Fajar	40	Umum	Rawat Jalan	200000.0
P007	Gina	22	Gigi	Rawat Jalan	180000.0

Message

Data berhasil ditambahkan!

OK

b. Pengujian Tampil Data

Pengujian dilakukan dengan menampilkan seluruh data pasien yang tersimpan pada array ke dalam tabel GUI.

Hasil:

Data pasien berhasil ditampilkan sesuai data yang telah diinput.

ID	Nama	Umur	Poli	Status	Total
P014	Nanda	19	Umum	Rawat Jalan	150000.0
P015	Oscar	55	Jantung	Rawat Inap	550000.0
P016	Putri	29	Anak	Rawat Jalan	200000.0
P017	Qori	37	THT	Rawat Inap	360000.0
P018	Rina	24	Gigi	Rawat Jalan	180000.0
P019	Sandi	48	Saraf	Rawat Inap	490000.0
P020	Tina	21	Kulit	Rawat Jalan	170000.0
P021	Udin	45	Jantung	Rawat Inap	520000.0
P022	Vina	31	Umum	Rawat Jalan	215000.0
P023	Wawan	52	THT	Rawat Inap	390000.0
P024	Xena	27	Anak	Rawat Jalan	200000.0
P025	Yusuf	39	Gigi	Rawat Inap	340000.0
P026	Zahra	23	Kulit	Rawat Jalan	185000.0
P050	Ramzi	19	Gigi	Aktif	4000000.0

c. Pengujian Hapus Data

Pengujian dilakukan dengan menghapus data menggunakan metode *soft delete*.

Hasil:

Data pasien berhasil dihapus dari tampilan tabel, tetapi data masih tersimpan di dalam array dengan status nonaktif.

Menu				
Hapus		Sort Nama		
Sort Total		Statistik		
ID	Nama	Umur	Poli	Status
P013	Mi	37	Saraf	Rawat Jalan
P014	Na	24	Umum	Rawat Jalan
P015	Os	48	Jantung	Rawat Jalan
P016	Pu	21	Anak	Rawat Jalan
P017	Qori	45	THT	Rawat Jalan
P018	Rina	31	Gigi	Rawat Jalan
P019	Sandi	52	Saraf	Rawat Jalan
P020	Tina	27	Kulit	Rawat Jalan
P021	Udin	39	Jantung	Rawat Jalan
P022	Vina	23	Umum	Rawat Jalan
P023	Wawan	37	THT	Rawat Jalan
P024	Xena	24	Anak	Rawat Jalan
P025	Yusuf	48	Gigi	Rawat Jalan
P026	Zahra	21	Kulit	Rawat Jalan

5.2 Skenario Pengujian Searching

a. Linear Search

Pengujian dilakukan dengan mencari data pasien berdasarkan ID menggunakan metode Linear Search.

Hasil:

Data pasien berhasil ditemukan sesuai kata kunci pencarian.

Pencarian					
P001				Cari Nama	Cari ID
Menu					
Hapus		Sort Nama		Sort ID	
Sort Total		Statistik		Refresh	
ID	Nama	Umur	Poli	Status	Total
P001	Andi	20	Umum	Rawat Jalan	150000.0

b. Binary Search

Pengujian dilakukan pada data yang sudah diurutkan terlebih dahulu menggunakan Binary Search.

Hasil:

Proses pencarian berjalan lebih cepat dan data berhasil ditemukan.

Zahra				Cari Nama	Cari ID
Menu					
Hapus		Sort Nama		Sort ID	
Sort Total		Statistik		Refresh	
ID	Nama	Umur	Poli	Status	Total
P026	Zahra	23	Kulit	Rawat Jalan	185000.0

5.3 Skenario Pengujian Sorting

Pengujian sorting dilakukan untuk melihat apakah data pasien dapat diurutkan dengan benar.

a. Bubble Sort

Data diurutkan berdasarkan nama atau ID pasien menggunakan Bubble Sort.

Hasil:

Data berhasil diurutkan sesuai urutan yang dipilih.

Hapus	
Sort Total	
ID	Nama
P001	Andi
P002	Budi
P003	Citra
P004	Dewi
P005	Eka
P006	Fajar
P007	Gina
P008	Hadi
P009	Intan
P010	Joko
P011	Kevin
P012	Lina
P013	Maya
P014	Nanda

b. Selection Sort

Data diurutkan menggunakan metode Selection Sort.

Hasil:

Data berhasil tersusun secara terurut sesuai proses sorting.

ID	Nama
P001	Andi
P027	Asep
P028	Bella
P002	Budi
P029	Cahyo
P003	Citra
P004	Dewi
P030	Dina
P005	Eka
P006	Fajar
P007	Gina
P008	Hadi
P009	Intan
P010	Joko

c. Sorting Data pada Tabel

Pengujian dilakukan dengan melihat hasil akhir pengurutan data pada tabel GUI.

Hasil:

Tampilan data menjadi lebih rapi dan mudah dicari.

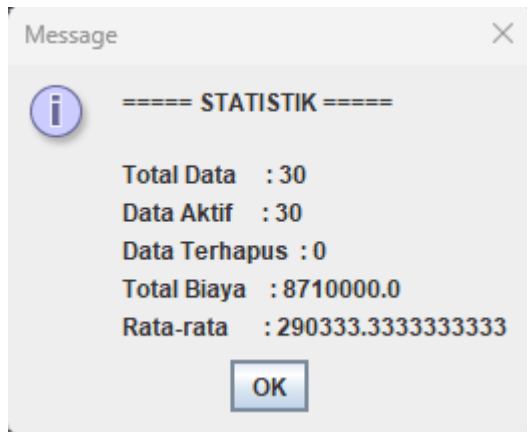
ID	Nama	Umur	Poli	Status	Total
P027	Asep	34	Saraf	Rawat Jalan	285000.0
P004	Dewi	28	Umum	Rawat Inap	280000.0
P013	Maya	41	Saraf	Rawat Jalan	270000.0
P005	Eka	35	Saraf	Rawat Jalan	260000.0
P011	Kevin	32	THT	Rawat Jalan	230000.0
P002	Budi	25	Gigi	Rawat Inap	220000.0
P022	Vina	31	Umum	Rawat Jalan	215000.0
P016	Putri	29	Anak	Rawat Jalan	200000.0
P024	Xena	27	Anak	Rawat Jalan	200000.0
P026	Zahra	23	Kulit	Rawat Jalan	185000.0
P003	Citra	30	Anak	Rawat Jalan	180000.0
P009	Intan	27	Anak	Rawat Jalan	180000.0
P018	Rina	24	Gigi	Rawat Jalan	180000.0
P030	Dina	20	Anak	Rawat Jalan	170000.0

5.4 Skenario Pengujian Statistik Data

Pengujian dilakukan untuk melihat jumlah pasien dan total pendapatan klinik berdasarkan data yang tersimpan.

Hasil:

Sistem berhasil menampilkan statistik data pasien dan total pendapatan dengan benar.



5.5 Skenario Pengujian Simpan dan Load File

Pengujian dilakukan untuk memastikan data pasien dapat disimpan dan dibuka kembali.

a. Simpan File

Data pasien disimpan ke dalam file menggunakan fitur save.

Hasil:

File berhasil tersimpan tanpa error.

b. Load File

Data yang sudah disimpan kemudian dibuka kembali ke dalam program.

Hasil:

Data berhasil dimuat kembali dan tampil pada tabel sistem.

BAB VI

KESIMPULAN

6.1 Kesimpulan

Berdasarkan hasil pembuatan dan pengujian program, Sistem Manajemen Pasien Klinik/Rumah Sakit telah berhasil diimplementasikan menggunakan bahasa pemrograman Java dengan antarmuka grafis Java Swing. Berikut adalah ringkasan pencapaian dari proyek ini:

Struktur Data: Sistem berhasil mengimplementasikan struktur data array of records (Pasien[]) untuk menyimpan dan mengelola data pasien dengan kapasitas hingga 200 entri. Penggunaan array memungkinkan akses $O(1)$ pada operasi penambahan data.

CRUD Operations: Seluruh operasi CRUD (Create, Read, Update, Delete) berhasil diimplementasikan dengan benar. Fitur soft delete dengan flag boolean aktif berhasil menjaga integritas data audit tanpa menghapus data secara fisik.

Algoritma Pencarian: Linear Search berhasil diimplementasikan untuk pencarian nama dengan partial match, memberikan fleksibilitas dalam pencarian. Pencarian berdasarkan ID mengimplementasikan prinsip exact match yang efisien.

Algoritma Pengurutan: Bubble Sort berhasil mengurutkan data berdasarkan ID secara ascending dengan $O(n^2)$ time complexity. Selection Sort berhasil mengurutkan data berdasarkan nama secara alfabetis. Keduanya diimplementasikan secara manual tanpa library eksternal.

Fitur Tambahan: Fitur statistik data, persistensi file I/O (simpan/muat data.txt), dan tampilan GUI yang intuitif berhasil ditambahkan sebagai nilai tambah sistem.

Validasi Input: Sistem mengimplementasikan error handling menggunakan try-catch untuk menangani input yang tidak valid, meningkatkan robustness program.

6.2 Perbandingan Efisiensi Algoritma

Dari analisis kompleksitas yang dilakukan, dapat disimpulkan bahwa untuk dataset berukuran sedang (30-200 data), perbedaan performa antara Linear Search $O(n)$ dan Binary Search $O(\log n)$ masih dapat diterima. Namun untuk dataset yang lebih besar, Binary Search akan memberikan keuntungan yang signifikan. Demikian pula, Bubble Sort dan Selection Sort sama-sama $O(n^2)$ namun cocok untuk dataset kecil yang umum ditemukan pada sistem klinik skala menengah.